# Discussion

# The Zero-Mock Tradeoff

## The Zero-Mock Tradeoff I

The Zero-Mock testing policy is `template/`'s most distinctive design decision. By prohibiting all mock objects, we gain confidence that tests exercise real code paths—a pytest run against the template genuinely invokes `pandoc`, writes to disk, and parses real YAML. The cost is test duration: the full infrastructure test suite (~12,546 tests) runs for minutes rather than the sub-second execution typical of heavily-mocked suites. This manuscript deliberately reports no wall-clock figure, consistent with the Results section's discipline of declining timing claims without a versioned benchmark artifact.

## The Zero-Mock Tradeoff I

We argue this tradeoff is strongly favorable for research software. Unlike web applications where millisecond latency and thousands of daily deploys demand fast feedback loops, research pipelines run infrequently (once per manuscript revision) and correctness vastly outweighs speed. A mocked test that passes while the real renderer fails is worse than a slow test that catches the failure. The analogy to statistical methodology is precise: just as Simmons et al.'s *researcher degrees of freedom* (Simmons et al. 2011) inflate false-positive rates through undisclosed analytical flexibility, mock objects create *testing degrees of freedom* that make integration failures invisible. The Zero-Mock policy closes this loophole by the same mechanism that pre-registration (Nosek et al. 2018) closes the p-hacking loophole: removing flexibility before the fact. As Peng (Peng 2011) argues, computational reproducibility requires independent

verification—and mock-only tests verify assumptions rather than results. Garijo et al.'s FAIRsoft evaluator (Garijo et al. 2024) identifies *executability* as a primary quality indicator; the Zero-Mock policy operationalizes executability at the unit level.

## When Mocks Are Not the Problem I

It is important to distinguish the Zero-Mock policy from a naive rejection of all test isolation techniques. Fowler's classification (Martin 2008) recognizes that stubs and fakes serve legitimate purposes—a test database populated with known data is not a mock, it is a fixture. The policy specifically prohibits *mock objects* as defined by Meszaros: assertions on indirect outputs (method calls, argument patterns) rather than direct outputs (return values, side effects). The distinction matters because mock-based assertions encode implementation assumptions ("method X must be called with argument Y") that become invisible coupling between tests and production code, creating the illusion of coverage without testing real behavior.

## Practical Implementation I

The template enforces zero-mock compliance at three levels:

# Practical Implementation I

1. **Code review**: `AGENTS.md` at every directory level explicitly states the prohibition, ensuring both human and AI contributors are aware before writing tests.
2. **Static analysis**: `scripts/audit/verify_no_mocks.py` implements the enforced lexical gate (prohibited mock-framework imports and calls) and an `--inventory` mode that separately classifies environment isolation from dependency-replacement debt; pre-push runs it automatically.
3. **Cultural norm**: `template_code_project` documents filesystem + YAML + plotting paths while `template_autoresearch_project` exercises readiness planning; `template_search_project` reinforces HTTP-realistic literature queries—both serve as onboarding references alongside infra suites.

However, the policy requires careful management of external dependencies. Tests requiring Ollama (the local LLM backend) use `@pytest.mark.requires_ollama` and are skipped in environments where the service is unavailable. Tests requiring network access use `@pytest.mark.network`. This marker system preserves the Zero-Mock principle while acknowledging that not all environments provide all services, especially computationally intensive ones. The key distinction is between *replacing* an external dependency (which mock objects do, hiding failures) and *skipping* a test when a dependency is absent (which markers do, preserving transparency).